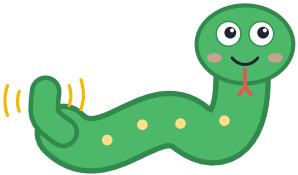
Recitation 3

Part 3: Functions and Dictionaries

Recitation 3: Build and Look Up

Write your own functions, store data by name, find and catch the bug



Dror Jacoby
Tel Aviv University

Where we are


L1
Hello, Python!
L2
Variables & Types
L3
Input & Output
L4
Making Decisions
L5
Lists & Sets
L6
Loops
L7
Functions
L8
Dicts & Debugging
L9
Objects, NumPy & Pandas
L10
Plots & ML
Today we practise L7–L8

Today's plan

◎ By the end of today you can...

- Wrap a job in a function and get the answer back
- Store and count things by name with a dictionary
- Walk through a table, find the middle, fix and catch the bug



Exercises marked **at home** are for practice after class. 🏠

Topic 1

Recap and warm-up

Recitation 3 • Functions & dictionaries



TODAY

1. **Recap and warm-up**
2. Worked problems
3. Your turn
4. Exam corner
5. Solutions

Lecture 7 in one slide: functions

```python
def make_coffee(size, sugar=0):
    return f"A {size} coffee, {sugar} sugars"

order = make_coffee("large")
print(order)
print(make_coffee("small", 2))
```

OUTPUT

```
A large coffee, 0 sugars
A small coffee, 2 sugars
```

- `def` builds the machine, a **call** runs it
- Values go in through **parameters**
- `return` hands the answer back
- `sugar=0`: a default, used if nothing is given

Lecture 7 in one slide: scope

Main program
variables made out here

Inside a function
parameters + variables made in here

erased when the function ends

- Inside **can read** outside
- Outside **cannot see** inside
- Same name? Two **different** boxes

Data goes in through parameters and comes out through return.

Lecture 8 in one slide: dictionaries

Write	Result
`menu = {"falafel": 18, "coffee": 12}`	key → value
`menu["coffee"]`	`12`
`menu["sabich"] = 24`	new key: added
`menu["coffee"] = 14`	old key: updated
`"tea" in menu`	`False` (checks the **keys**)
`menu.get("tea", 0)`	`0` instead of a crash
`for item, price in menu.items():`	each key **with** its value
`counts[x] = counts.get(x, 0) + 1`	the tally recipe



Lists find by position. Dicts find by name.

Lecture 8 in one slide: tables, middle, bugs

```
seats = [["Dana", "Omer", "Lior"],
         ["Noa", "Yael", "Tom"]]
print(seats[1][2])
print(sorted([5, 1, 9, 3, 7])[5 // 2])
```

OUTPUT

```
Tom
5
```

Row first, then column. Median: sort, take the middle.

Error	In plain words
`NameError`	never heard of that name
`TypeError`	these types don’t mix
`IndexError`	no item at that position
`KeyError`	no such key

Read the error from the **bottom** up. No error but a wrong answer? **Print** the boxes.

Warm-up 1: print or return?

Predict the output!

```
def double(x):
    return x * 2

def shout(x):
    print(x * 2)

a = double(4)
b = shout(4)
print(a, b)
```



Warm-up 1: print or return?

Predict the output!

```
def double(x):
    return x * 2

def shout(x):
    print(x * 2)

a = double(4)
b = shout(4)
print(a, b)
```

Answer

8
8 None

`shout` prints 8 right away, but returns nothing: `b` is `None`.



Warm-up 2: whose score?

Predict the output!

```
score = 10

def bonus(score):
    score = score + 5
    return score

print(bonus(score))
print(score)
```



Warm-up 2: whose score?

Predict the output!

```
score = 10

def bonus(score):
    score = score + 5
    return score

print(bonus(score))
print(score)
```

Answer

15
10

The parameter `score` is the function's **own** box. The outside `score` is still 10.



Warm-up 3: the cinema ticket

Predict the output!

```
def ticket(age, price=40):
    if age < 18:
        return price / 2
    return price

print(ticket(30))
print(ticket(12))
print(ticket(70, 25))
```

Warm-up 3: the cinema ticket

Predict the output!

```
def ticket(age, price=40):
    if age < 18:
        return price / 2
    return price

print(ticket(30))
print(ticket(12))
print(ticket(70, 25))
```

Answer

```
40
20.0
25
```

`/` always gives a float. `return` ends the function, so no `else` is needed.

Warm-up 4: the corner store

Predict the output!

```
stock = {"apples": 12, "milk": 3}
stock["milk"] = stock["milk"] - 1
stock["bread"] = 5
print(stock)
print(stock.get("eggs", 0), "eggs" in stock)
print(len(stock))
```

Warm-up 4: the corner store

Predict the output!

```
stock = {"apples": 12, "milk": 3}
stock["milk"] = stock["milk"] - 1
stock["bread"] = 5
print(stock)
print(stock.get("eggs", 0), "eggs" in stock)
print(len(stock))
```

Answer

```
{'apples': 12,
'milk': 2, 'bread': 5}
0 False
3
```

`len` counts the keys.

Warm-up 5: row, then column

Predict the output!

```
board = [[1, 2, 3],
         [4, 5, 6]]
print(board[1][0])
total = 0
for row in board:
    total = total + row[-1]
print(total)
```



Warm-up 5: row, then column

Predict the output!

```
board = [[1, 2, 3],
         [4, 5, 6]]
print(board[1][0])
total = 0
for row in board:
    total = total + row[-1]
print(total)
```

Answer

4
9

Row 1 is `[4, 5, 6]`, its item 0 is 4. `row[-1]` is the last item of each row: $3 + 6 = 9$.



Warm-up 6: which line runs?

Predict the output!

```
for text in ["5", "five", "0"]:
    try:
        n = int(text)
        print("got", n)
        print(10 // n)
    except ValueError:
        print("bad")
    except ZeroDivisionError:
        print("zero")
```

Warm-up 6: which line runs?

Predict the output!

```
for text in ["5", "five", "0"]:
    try:
        n = int(text)
        print("got", n)
        print(10 // n)
    except ValueError:
        print("bad")
    except ZeroDivisionError:
        print("zero")
```

Answer

```
got 5
2
bad
got 0
zero
```

`int("five")` fails at once: no "got". `int("0")` works, so "got 0" prints, then `10 // 0` fails.

Topic 2

Worked problems

Recitation 3 • Functions & dictionaries



TODAY

1. Recap and warm-up
2. **Worked problems**
3. Your turn
4. Exam corner
5. Solutions

Problem 1: the course grade

The rules from the syllabus

40% homework + 60% exam, rounded. Exam under 56? The final grade is just the exam. 56+ passes.

Problem 1: the course grade

The rules from the syllabus

40% homework + 60% exam, rounded. Exam under 56? The final grade is just the exam. 56+ passes.



Plan in words first:

1. A function that **returns** the final grade
2. A function that answers: passed or not?
3. Use both on every student

Build it: two small functions

course_grade.py

```
def final_grade(hw, exam):
    if exam < 56:
        return exam
    return round(0.4 * hw + 0.6 * exam)

def passed(grade):
    return grade >= 56

print(final_grade(90, 80))
print(final_grade(100, 50))
print(passed(final_grade(70, 60)))
```

Build it: two small functions

course_grade.py

```
def final_grade(hw, exam):
    if exam < 56:
        return exam
    return round(0.4 * hw + 0.6 * exam)

def passed(grade):
    return grade >= 56

print(final_grade(90, 80))
print(final_grade(100, 50))
print(passed(final_grade(70, 60)))
```

OUTPUT

```
84
50
True
```

Build it: two small functions

course_grade.py

```
def final_grade(hw, exam):
    if exam < 56:
        return exam
    return round(0.4 * hw + 0.6 * exam)

def passed(grade):
    return grade >= 56

print(final_grade(90, 80))
print(final_grade(100, 50))
print(passed(final_grade(70, 60)))
```

OUTPUT

```
84
50
True
```

$0.4 \cdot 90 + 0.6 \cdot 80 = 84$.
passed returns `True` or `False`, so it fits right into an `if`.

Use them on the whole class

course_grade.py (continued)

```
students = {"Noa": [90, 80], "Itay": [100, 50],
            "Maya": [60, 58]}
for name, marks in students.items():
    grade = final_grade(marks[0], marks[1])
    if passed(grade):
        print(f"{name}: {grade} passed")
    else:
        print(f"{name}: {grade} failed")
```

Use them on the whole class

course_grade.py (continued)

```
students = {"Noa": [90, 80], "Itay": [100, 50],
            "Maya": [60, 58]}
for name, marks in students.items():
    grade = final_grade(marks[0], marks[1])
    if passed(grade):
        print(f"{name}: {grade} passed")
    else:
        print(f"{name}: {grade} failed")
```

OUTPUT

```
Noa: 84 passed
Itay: 50 failed
Maya: 59 passed
```

Use them on the whole class

course_grade.py (continued)

```
students = {"Noa": [90, 80], "Itay": [100, 50],
            "Maya": [60, 58]}
for name, marks in students.items():
    grade = final_grade(marks[0], marks[1])
    if passed(grade):
        print(f"{name}: {grade} passed")
    else:
        print(f"{name}: {grade} failed")
```

OUTPUT

```
Noa: 84 passed
Itay: 50 failed
Maya: 59 passed
```

A dict of name → `[homework, exam]`. Itay's 100 in homework does not help: the exam is under 56.

What if we forget one return?

What is printed?

```
def final_grade(hw, exam):
    if exam < 56:
        return exam
    grade = round(0.4 * hw + 0.6 * exam)

print(final_grade(90, 80))
print(final_grade(100, 50))
```

What if we forget one return?

What is printed?

```
def final_grade(hw, exam):
    if exam < 56:
        return exam
    grade = round(0.4 * hw + 0.6 * exam)

print(final_grade(90, 80))
print(final_grade(100, 50))
```

Answer

None
50

Noa's grade is computed, then thrown away. Only one road has a `return`.

Every road out of a function needs a return.

Problem 2: my Rav-Kav month

Your bus card's history

This month: lines 5, 18, 5, 72, 18, 5, 5, 72, 18, 5.
Rides per line? Which line is your top line?

Problem 2: my Rav-Kav month

Your bus card's history

This month: lines 5, 18, 5, 72, 18, 5, 5, 72, 18, 5.
Rides per line? Which line is your top line?



Plan in words:

1. A **tally board**: line → number of rides
2. Max recipe over the board: keep the best so far
3. Print the winner

Code it: count, then find the top

rav_kav.py

```
rides = ["5", "18", "5", "72", "18", "5",
         "5", "72", "18", "5"]
counts = {}
for line in rides:
    counts[line] = counts.get(line, 0) + 1
print(counts)

top = rides[0]
for line, times in counts.items():
    if times > counts[top]:
        top = line
print(f"Top: line {top}, {counts[top]} rides")
```

Code it: count, then find the top

rav_kav.py

```
rides = ["5", "18", "5", "72", "18", "5",
         "5", "72", "18", "5"]
counts = {}
for line in rides:
    counts[line] = counts.get(line, 0) + 1
print(counts)

top = rides[0]
for line, times in counts.items():
    if times > counts[top]:
        top = line
print(f"Top: line {top}, {counts[top]} rides")
```

OUTPUT

```
{'5': 5, '18': 3, '72': 2}
Top: line 5, 5 rides
```

Code it: count, then find the top

rav_kav.py

```
rides = ["5", "18", "5", "72", "18", "5",
         "5", "72", "18", "5"]
counts = {}
for line in rides:
    counts[line] = counts.get(line, 0) + 1
print(counts)

top = rides[0]
for line, times in counts.items():
    if times > counts[top]:
        top = line
print(f"Top: line {top}, {counts[top]} rides")
```

OUTPUT

```
{'5': 5, '18': 3, '72': 2}
Top: line 5, 5 rides
```

Line numbers are **names** here, not math, so we keep them as text.

What if we ask about a line we never took?

What happens?

```
counts = {"5": 5, "18": 3, "72": 2}
print(counts.get("4", 0))
print(counts["4"])
```

What if we ask about a line we never took?

What happens?

```
counts = {"5": 5, "18": 3, "72": 2}
print(counts.get("4", 0))
print(counts["4"])
```

OUTPUT

```
0
```

ERROR

```
KeyError: '4'
```

`get` gives a backup. Square brackets crash on a missing key.

Problem 3: the step challenge

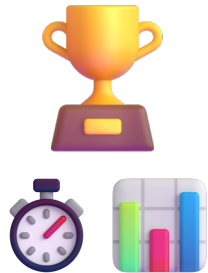
Three friends, five days

Each friend's steps per day go in one **row**. Who walked the most? Omer had one huge day: what is his "typical" day?

Problem 3: the step challenge

Three friends, five days

Each friend's steps per day go in one **row**. Who walked the most? Omer had one huge day: what is his "typical" day?



Plan in words:

1. A table: a list of rows
2. A loop inside a loop: total per row
3. Average vs **median** for Omer

Code it: one row per friend

steps.py

```
steps = [[6200, 8100, 4300, 12000, 7600],   # Dana
         [3000, 2500, 4100, 3800, 15000],   # Omer
         [9100, 8800, 9500, 9900, 8700]]    # Lior
names = ["Dana", "Omer", "Lior"]
for i in range(len(steps)):
    total = 0
    for day in steps[i]:
        total = total + day
    print(f"{names[i]}: {total} steps")
```

Code it: one row per friend

steps.py

```
steps = [[6200, 8100, 4300, 12000, 7600],   # Dana
         [3000, 2500, 4100, 3800, 15000],   # Omer
         [9100, 8800, 9500, 9900, 8700]]    # Lior
names = ["Dana", "Omer", "Lior"]
for i in range(len(steps)):
    total = 0
    for day in steps[i]:
        total = total + day
    print(f"{names[i]}: {total} steps")
```

OUTPUT

```
Dana: 38200 steps
Omer: 28400 steps
Lior: 46000 steps
```

Code it: one row per friend

steps.py

```
steps = [[6200, 8100, 4300, 12000, 7600],   # Dana
         [3000, 2500, 4100, 3800, 15000],   # Omer
         [9100, 8800, 9500, 9900, 8700]]    # Lior
names = ["Dana", "Omer", "Lior"]
for i in range(len(steps)):
    total = 0
    for day in steps[i]:
        total = total + day
    print(f"{names[i]}: {total} steps")
```

OUTPUT

```
Dana: 38200 steps
Omer: 28400 steps
Lior: 46000 steps
```

`total = 0` sits **inside** the outer loop: a fresh start for every friend.

What if...? Omer's typical day

```
def median(values):
    ordered = sorted(values)
    middle = len(ordered) // 2
    if len(ordered) % 2 == 1:
        return ordered[middle]
    else:
        return (ordered[middle - 1] + ordered[middle]) / 2

omer = [3000, 2500, 4100, 3800, 15000]
print(sum(omer) / len(omer))
print(median(omer))
```

What if...? Omer's typical day

```python
def median(values):
    ordered = sorted(values)
    middle = len(ordered) // 2
    if len(ordered) % 2 == 1:
        return ordered[middle]
    else:
        return (ordered[middle - 1] + ordered[middle]) / 2

omer = [3000, 2500, 4100, 3800, 15000]
print(sum(omer) / len(omer))
print(median(omer))
```

OUTPUT

```
5680.0
3800
```

What if...? Omer's typical day

```
def median(values):
    ordered = sorted(values)
    middle = len(ordered) // 2
    if len(ordered) % 2 == 1:
        return ordered[middle]
    else:
        return (ordered[middle - 1] + ordered[middle]) / 2

omer = [3000, 2500, 4100, 3800, 15000]
print(sum(omer) / len(omer))
print(median(omer))
```

OUTPUT

```
5680.0
3800
```

One 15,000-step hike pulls the average up. On a typical day Omer walks about 3,800.

What if...? Where does a day stand?

```
def quantile(values, q):
    ordered = sorted(values)
    position = int(q * (len(ordered) - 1))
    return ordered[position]

week = [6200, 8100, 4300, 12000, 7600,
        3000, 2500, 4100, 3800, 15000,
        9100, 8800, 9500, 9900, 8700]
print(quantile(week, 0.25))
print(quantile(week, 0.75))
```

What if...? Where does a day stand?

```
def quantile(values, q):
    ordered = sorted(values)
    position = int(q * (len(ordered) - 1))
    return ordered[position]

week = [6200, 8100, 4300, 12000, 7600,
        3000, 2500, 4100, 3800, 15000,
        9100, 8800, 9500, 9900, 8700]
print(quantile(week, 0.25))
print(quantile(week, 0.75))
```

OUTPUT

```
4100
9100
```

What if...? Where does a day stand?

```
def quantile(values, q):
    ordered = sorted(values)
    position = int(q * (len(ordered) - 1))
    return ordered[position]

week = [6200, 8100, 4300, 12000, 7600,
        3000, 2500, 4100, 3800, 15000,
        9100, 8800, 9500, 9900, 8700]
print(quantile(week, 0.25))
print(quantile(week, 0.75))
```

OUTPUT

```
4100
9100
```

All 15 days in one list. A quarter of the days are at or below 4,100, a quarter at or above 9,100.

Topic 3

Your turn

Recitation 3 • Functions & dictionaries



TODAY

1. Recap and warm-up
2. Worked problems
3. **Your turn**
4. Exam corner
5. Solutions

Your turn: 11 exercises

Exercise	Lecture	Level	Where
1 Fix the tip calculator	L7	★	in class
2 Bigger, and even?	L7	★★	in class
3 Weather in Fahrenheit	L7	★★	at home
4 The phone book	L8	★	in class
5 Count the words	L8	★★	in class
6 Order at the stand	L8	★★	at home
7 Tic-tac-toe board	L8	★	in class
8 Fix the median	L8	★★	in class
9 Average per student	L7-8	★★	at home
10 Library late fees	L7-8	★★★	at home
11 The safe divider	L7-8	★★	in class



Predict the output *before* you run.
Stuck for 3 minutes? Ask!

Exercise 1 ⭐: fix the tip calculator

EXERCISE 1

This code has 3 bugs. Fix them all.

```
def tip(bill, percent=10)
    print(bill * percent / 100)

print(tip(200))
print(tip(150 15))
```

Wanted output: 🪙

OUTPUT

```
20.0
22.5
```

Exercise 1 ⭐: fix the tip calculator

EXERCISE 1

This code has 3 bugs. Fix them all.

```
def tip(bill, percent=10)
    print(bill * percent / 100)

print(tip(200))
print(tip(150 15))
```

Wanted output: 🪙

OUTPUT

```
20.0
22.5
```

Hint

Two bugs crash. The third prints None: hand the tip **back**.

Exercise 2 ⭐⭐: bigger, and even?

EXERCISE 2

fill in the yellow blanks

```
____ bigger(a, b):
    if a > b:
        _____ a
    return ____

def is_even(n):
    return n % 2 ____ 0

print(bigger(7, 12))
print(is_even(7), is_even(12))
```

Wanted output: 🧩

OUTPUT

```
12
False True
```

Exercise 2 ⭐⭐: bigger, and even?

EXERCISE 2

fill in the yellow blanks

```
____ bigger(a, b):
    if a > b:
        _____ a
    return ____

def is_even(n):
    return n % 2 ____ 0

print(bigger(7, 12))
print(is_even(7), is_even(12))
```

Wanted output: 🧩

OUTPUT

```
12
False True
```

Hint

A comparison is already `True` or `False`: return it.

Exercise 3 ⭐⭐: weather in Fahrenheit

AT HOME 3

Write it from scratch

- A function `c_to_f(celsius)` that **returns** $c \cdot 9/5 + 32$
- Loop over `[18, 24, 31, 15]`, print each day
- Over 80 F? Add `hot!`

Wanted output: 🌡️

OUTPUT

```
18C = 64.4F
24C = 75.2F
31C = 87.8F  hot!
15C = 59.0F
```

Exercise 3 ⭐⭐: weather in Fahrenheit

AT HOME 3

Write it from scratch

- A function `c_to_f(celsius)` that **returns** $c \cdot 9/5 + 32$
- Loop over `[18, 24, 31, 15]`, print each day
- Over 80 F? Add `hot!`

Wanted output: 🌡️

OUTPUT

```
18C = 64.4F
24C = 75.2F
31C = 87.8F  hot!
15C = 59.0F
```

Hint

The function only converts. The loop and the `if` stay outside.

Exercise 4 ⭐: the phone book

EXERCISE 4

fill in the dictionary tools

```
phones = {"Noa": "050-1234567",
          "Omer": "052-7654321"}
phones[          ] = "054-1112223"
print(    (phones))
print(phones.    ("Yossi", "not saved"))
print("Dana"   phones)
```

Wanted output: 📱

OUTPUT

```
3
not saved
True
```

Exercise 4 ⭐: the phone book

EXERCISE 4

fill in the dictionary tools

```
phones = {"Noa": "050-1234567",
          "Omer": "052-7654321"}
phones[          ] = "054-1112223"
print(     (phones))
print(phones.     ("Yossi", "not saved"))
print("Dana"    phones)
```

Wanted output:

OUTPUT

```
3
not saved
True
```

Hint

Add Dana, count, look up safely, check.

Exercise 5 ⭐⭐: count the words

EXERCISE 5

fill in the tally recipe

```
text = "to be or not to be"
counts = {}
for word in text.split():
    counts[word] = counts.get(word, 0) + 1
print(counts)
```

Wanted output:

OUTPUT

```
{'to': 2, 'be': 2,
 'or': 1, 'not': 1}
```

Exercise 5 ⭐⭐: count the words

EXERCISE 5

fill in the tally recipe

```
text = "to be or not to be"
counts = {}
for word in text.split():
    counts[word] = counts.get(word, 0) + 1
print(counts)
```

Wanted output: abc

OUTPUT

```
{'to': 2, 'be': 2,
 'or': 1, 'not': 1}
```

Hint

Start with an empty board. `split` cuts the text into words.

Exercise 6 ⭐⭐: order at the stand

AT HOME 6

Write it from scratch with a dictionary

Menu: falafel 18, sabich 24, coffee 12.
Order: falafel, coffee, pizza, falafel.

Add up the order. Not on the menu? Say sorry and skip it.

Wanted output: 🫐

OUTPUT

```
Sorry, no pizza
Total: 48 ILS
```

Exercise 6 ⭐⭐: order at the stand

AT HOME 6

Write it from scratch with a dictionary

Menu: falafel 18, sabich 24, coffee 12.
Order: falafel, coffee, pizza, falafel.

Add up the order. Not on the menu? Say sorry and skip it.

Wanted output: 

OUTPUT

```
Sorry, no pizza
Total: 48 ILS
```

Hint

Before a lookup, ask: is the item `in` the menu?

Exercise 7 ⭐: tic-tac-toe board

EXERCISE 7

fill in the positions

```
board = [["X", "O", " "],
         [" ", "X", "O"],
         ["O", " ", " "]]
print(board[   ][   ])     # the middle
board[   ][   ] = "X"      # bottom right
x_count = 0
for row in board:
    for cell in      :
        if cell == "X":
            x_count = x_count + 1
print(f"X has {x_count} marks")
```

Wanted output: 🎮

OUTPUT

```
X
X has 3 marks
```

Exercise 7 ⭐: tic-tac-toe board

EXERCISE 7

fill in the positions

```
board = [["X", "O", " "],
         [" ", "X", "O"],
         ["O", " ", " "]]
print(board[   ][   ])     # the middle
board[   ][   ] = "X"      # bottom right
x_count = 0
for row in board:
    for cell in      :
        if cell == "X":
            x_count = x_count + 1
print(f"X has {x_count} marks")
```

Wanted output: 🎮

OUTPUT

```
X
X has 3 marks
```

Hint

Rows and columns both start at 0. The bottom row is row 2.

Exercise 8 ⭐⭐: fix the median

EXERCISE 8

3 bugs. Fix them.

```
def median(values):
    middle = len(values) / 2
    return values[middle]

print(median([30, 10, 20]))
print(median([30, 10, 20, 40]))
```

It crashes: `TypeError: list indices must be integers or slices, not float`

Wanted output: 📊

OUTPUT

```
20
25.0
```

Exercise 8 ⭐⭐: fix the median

EXERCISE 8

3 bugs. Fix them.

```
def median(values):
    middle = len(values) / 2
    return values[middle]

print(median([30, 10, 20]))
print(median([30, 10, 20, 40]))
```

It crashes: `TypeError: list indices must be integers or slices, not float`

Wanted output:

OUTPUT

```
20
25.0
```

Hint

Fix the crash first. Then: was the list sorted? What if the count is even?

Exercise 9 ⭐⭐: average per student

AT HOME 9

✏️ Write it from scratch

Write `row_average(row)` with a loop (no `sum`). Use it on every row of this table:

```
grades = [[90, 85, 70],
          [60, 75, 81],
          [100, 95, 88]]
names = ["Dana", "Omer", "Lior"]
```

Wanted output: 🎓

OUTPUT

```
Dana: 81.7
Omer: 72.0
Lior: 94.3
```

Exercise 9 ⭐⭐: average per student

AT HOME 9

✏ Write it from scratch

Write `row_average(row)` with a loop (no `sum`). Use it on every row of this table:

```
grades = [[90, 85, 70],
          [60, 75, 81],
          [100, 95, 88]]
names = ["Dana", "Omer", "Lior"]
```

Wanted output: 🎓

OUTPUT

```
Dana: 81.7
Omer: 72.0
Lior: 94.3
```

💡 Hint

One decimal: `{avg:.1f}`. Loop over the positions with `range(len(grades))`.

Exercise 10 ⭐⭐⭐: library late fees

AT HOME 10

✏️ Write it from scratch

`late_fee(days)`: 14 days free, then 2 ILS a day.

Loans: Noa 10 days, Itay 21, Maya 30, as a **list of dictionaries** with keys `"name"` and `"days"`.

Print each fee, the total, and who owes the most.

Wanted output: 📚

OUTPUT

```
Noa: 0 ILS
Itay: 14 ILS
Maya: 32 ILS
Total fees: 46 ILS
Owes the most: Maya
```

Exercise 10 ⭐⭐⭐: library late fees

AT HOME 10

Write it from scratch

`late_fee(days)`: 14 days free, then 2 ILS a day.

Loans: Noa 10 days, Itay 21, Maya 30, as a **list of dictionaries** with keys `"name"` and `"days"`.

Print each fee, the total, and who owes the most.

Wanted output: 📚

OUTPUT

```
Noa: 0 ILS
Itay: 14 ILS
Maya: 32 ILS
Total fees: 46 ILS
Owes the most: Maya
```

Hint

One loop, three jobs: fee, total, max.

Exercise 11 ⭐⭐: the safe divider

EXERCISE 11

It crashes on bad input. Make it safe.

```
def safe_divide(a_text, b_text):
    return float(a_text) / float(b_text)

print(safe_divide("12", "4"))
print(safe_divide("12", "abc"))
print(safe_divide("12", "0"))
```

Wanted output: 🛡️

OUTPUT

```
3.0
Not a number!
Can't divide by 0!
```

Bonus: ask the user for two numbers with `input`, again and again until they work.

Exercise 11 ⭐⭐: the safe divider

EXERCISE 11

It crashes on bad input. Make it safe.

```
def safe_divide(a_text, b_text):
    return float(a_text) / float(b_text)

print(safe_divide("12", "4"))
print(safe_divide("12", "abc"))
print(safe_divide("12", "0"))
```

Bonus: ask the user for two numbers with `input`, again and again until they work.

Wanted output:

OUTPUT

```
3.0
Not a number!
Can't divide by 0!
```

Hint

Two errors, two `except` blocks. Read each error's last line for its name.

Your home practice list

Before the next recitation

- Exercises 3, 6, 9 and 10 **at home**
- Any in-class exercise you did not finish
- Then check yourself against the solutions

Type the code yourself. Reading a solution is not practice.



Topic 4

Exam corner

Recitation 3 • Functions & dictionaries



TODAY

1. Recap and warm-up
2. Worked problems
3. Your turn
4. **Exam corner**
5. Solutions

Exam style 1: trace the call

What is printed?

```
def change(n):
    n = n + 1
    print("in:", n)
    return n * 2

n = 3
m = change(n)
print(n, m)
```

Exam style 1: trace the call

What is printed?

```
def change(n):
    n = n + 1
    print("in:", n)
    return n * 2

n = 3
m = change(n)
print(n, m)
```

Answer

```
in: 4
3 8
```

Inside, `n` is a copy: 3 becomes 4 and the function returns 8. The outside `n` stays 3.

Exam tip: draw an “outside” and an “inside” box.

Exam style 2: trace the dictionary

What is printed?

```
prices = {"tea": 8, "cake": 15, "juice": 11}
total = 0
for item, price in prices.items():
    if price > 10:
        total = total + price
    else:
        prices[item] = price + 2
print(total, prices["tea"])
```

Exam style 2: trace the dictionary

What is printed?

```
prices = {"tea": 8, "cake": 15, "juice": 11}
total = 0
for item, price in prices.items():
    if price > 10:
        total = total + price
    else:
        prices[item] = price + 2
print(total, prices["tea"])
```

Answer

26 10

item	price	total
tea	8	0
cake	15	15
juice	11	26

Tea: not over 10, so it goes up to 10.

Exam style 3: the diagonal

What is printed?

```
grid = [[1, 0, 2],
        [3, 1, 0],
        [0, 4, 1]]
count = 0
for i in range(3):
    if grid[i][i] == 1:
        count = count + 1
print(count, grid[2][1] + grid[0][2])
```



Exam style 3: the diagonal

What is printed?

```
grid = [[1, 0, 2],
        [3, 1, 0],
        [0, 4, 1]]
count = 0
for i in range(3):
    if grid[i][i] == 1:
        count = count + 1
print(count, grid[2][1] + grid[0][2])
```



Answer

3 6

`grid[i][i]`: same row and column, so the diagonal. All three are 1. Then $4 + 2 = 6$.

Three new mistakes to avoid

```
def has_negative(numbers):
    for num in numbers:
        if num < 0:
            return True
        return False

print(has_negative([4, 7, -2]))
```

OUTPUT

```
False
```

`return False` inside the loop ends it after the **first** number. Move it out of the loop.

```
def greet(name):
    return f"Hi {name}!"

print(greet())
```

ERROR

```
TypeError: greet() missing 1 required positional argument: 'name'
```

One parameter, one value in the call.

```
menu = {"falafel": 18,
        "sabich": 24}
print(menu["Falafel"])
```

ERROR

```
KeyError: 'Falafel'
```

Keys must match **exactly**, capital letters too.

Before the homework

⚠ HW2 now, HW3 next

HW2 (Functions and Loops) is due ∼Dec 20 on Moodle.
HW3 is the final project. It comes in January: dictionaries and tables from today, plus classes and the data tools of Lectures 9–10.



- Small functions, each with one job
- Test each function alone before you combine them

Recap

✔ Today you learned

- In through parameters, out through `return`. Every road needs one
- What happens in the function stays in the function
- Dicts look up by key: `get` and `in` avoid the `KeyError`
- Tables: row first, then column. Median: sort, take the middle
- `try` / `except`: plan for bad input



Topic 5

Solutions

Recitation 3 • Functions & dictionaries



TODAY

1. Recap and warm-up
2. Worked problems
3. Your turn
4. Exam corner
5. **Solutions**

Solution 1: fix the tip calculator

SOLUTION

```
def tip(bill, percent=10):
    return bill * percent / 100

print(tip(200))
print(tip(150, 15))
```

OUTPUT

```
20.0
22.5
```

- A colon `:` after the `def` line
- `return`, not `print`: otherwise `print(tip(200))` shows `None`
- A comma between the two values

Solution 2: bigger, and even?

SOLUTION

```
def bigger(a, b):
    if a > b:
        return a
    return b

def is_even(n):
    return n % 2 == 0
```

OUTPUT

```
12
False True
```

If `a > b` we leave with `a`. Otherwise we reach the last line and return `b`: no `else` needed.

`n % 2 == 0` is already `True` or `False`.

Solution 3: weather in Fahrenheit

SOLUTION

```
def c_to_f(celsius):
    return celsius * 9 / 5 + 32

week = [18, 24, 31, 15]
for c in week:
    f = c_to_f(c)
    if f > 80:
        print(f"{c}C = {f}F  hot!")
    else:
        print(f"{c}C = {f}F")
```

OUTPUT

```
18C = 64.4F
24C = 75.2F
31C = 87.8F  hot!
15C = 59.0F
```

The function does one job. We call it once per day and keep the answer in `f`.

Solutions 4 and 5: phone book and word cou

SOLUTION

Exercise 4 A new key in square brackets adds it.

```
phones["Dana"] = "054-1112223"
print(len(phones))
print(phones.get("Yossi", "not saved"))
print("Dana" in phones)
```

Exercise 5 `get(word, 0)`: the count so far, or 0 for a new word.

```
counts = {}
for word in text.split():
    counts[word] = counts.get(word, 0) + 1
```

Solution 6: order at the stand

SOLUTION

```
menu = {"falafel": 18, "sabich": 24, "coffee": 12}
order = ["falafel", "coffee", "pizza", "falafel"]
total = 0
for item in order:
    if item in menu:
        total = total + menu[item]
    else:
        print(f"Sorry, no {item}")
print(f"Total: {total} ILS")
```

OUTPUT

```
Sorry, no pizza
Total: 48 ILS
```

$18 + 12 + 18 = 48$. Without the `in` check, `menu["pizza"]` would crash with a `KeyError`.

Solution 7: tic-tac-toe board

SOLUTION

```
print(board[1][1])      # the middle
board[2][2] = "X"       # bottom right
x_count = 0
for row in board:
    for cell in row:
        if cell == "X":
            x_count = x_count + 1
```

OUTPUT

```
X
X has 3 marks
```

The board had 2 X marks, plus the new one in the bottom right.

Solution 8: fix the median

SOLUTION

```
def median(values):
    ordered = sorted(values)
    middle = len(ordered) // 2
    if len(ordered) % 2 == 1:
        return ordered[middle]
    return (ordered[middle - 1] + ordered[middle]) / 2

print(median([30, 10, 20]))
print(median([30, 10, 20, 40]))
```

OUTPUT

```
20
25.0
```

- `//`, not `/`: a position must be a whole number
- Sort first
- Even count: average the two middle values

Solution 9: average per student

SOLUTION

```python
def row_average(row):
    total = 0
    for value in row:
        total = total + value
    return total / len(row)

grades = [[90, 85, 70],
          [60, 75, 81],
          [100, 95, 88]]
names = ["Dana", "Omer", "Lior"]
for i in range(len(grades)):
    print(f"{names[i]}: {row_average(grades[i]):.1f}")
```

OUTPUT

```
Dana: 81.7
Omer: 72.0
Lior: 94.3
```

The loop inside the loop is now hidden in the function. Each row is just a list.

Solution 10: library late fees

SOLUTION

```
loans = [{"name": "Noa", "days": 10},
         {"name": "Itay", "days": 21},
         {"name": "Maya", "days": 30}]
total = 0
worst = loans[0]
for loan in loans:
    fee = late_fee(loan["days"])
    total = total + fee
    if loan["days"] > worst["days"]:
        worst = loan
    print(f"{loan['name']}: {fee} ILS")
print(f"Total fees: {total} ILS")
print(f"Owes the most: {worst['name']}")
```

```
def late_fee(days):
    if days <= 14:
        return 0
    return (days - 14) * 2
```

OUTPUT

```
Noa: 0 ILS
Itay: 14 ILS
Maya: 32 ILS
Total fees: 46 ILS
Owes the most: Maya
```

3D emoji: Microsoft Fluent Emoji (MIT License). Pip the snake: original drawing. Full list: images/CREDITS.md

Solution 11: the safe divider

SOLUTION

```python
def safe_divide(a_text, b_text):
    try:
        return float(a_text) / float(b_text)
    except ValueError:
        return "Not a number!"
    except ZeroDivisionError:
        return "Can't divide by 0!"
print(safe_divide("12", "4"))
print(safe_divide("12", "abc"))
print(safe_divide("12", "0"))
```

OUTPUT

```
3.0
Not a number!
Can't divide by 0!
```

- `float("abc")`: `ValueError`
- `12 / 0.0`: `ZeroDivisionError`
- Each **`except` returns** a message
- Never a bare **`except`**: